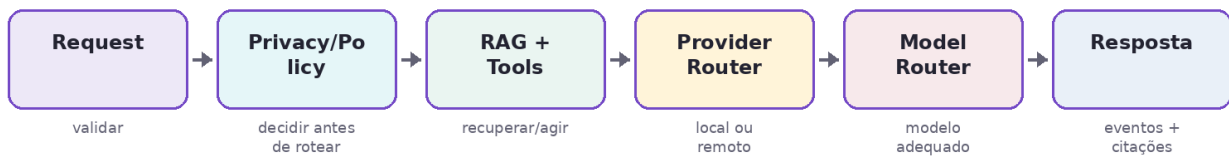


ATENA

LABORATÓRIO PRÁTICO DE PROGRAMAÇÃO

Do Linux Mint e Geany a uma IA nativa, local, híbrida e integrável ao Athenas OS

Fluxo prático da Atena mínima utilizável



Livro de bancada • edição de projeto 0.1 • agosto de 2026

Ambiente principal: Linux Mint • Editor: Geany • Linguagem: C17 • Build: CMake • Runtime local: llama.cpp/llama-server

PROJETO ODISSEIA | ATHENAS OS

Como usar este livro

Este livro é deliberadamente diferente de um manual teórico. Ele foi escrito como uma sequência de sessões práticas de bancada: você lê o objetivo, executa comandos no Linux Mint, abre os arquivos no Geany, compila, testa, observa o resultado e só então avança. Cada conceito de C aparece quando passa a ser necessário para a Atena. Não há uma etapa de “estudar C inteiro” antes de programar a IA.

A meta não é construir um modelo de linguagem do zero. A meta é programar a plataforma Atena: o Core, os contratos, os providers, a política de privacidade, a memória, o RAG, as ferramentas, o roteamento, o serviço de usuário e a integração com o Athenas OS. O modelo GGUF é um motor cognitivo substituível dentro dessa plataforma.

REGRA DE BANCADA: Execute os laboratórios em ordem. Quando um checkpoint não passar, não “compense” o erro copiando comandos adiante. Corrija a camada atual; as próximas dependem dela.

Símbolo mental	O que significa	O que fazer
TERMINAL	comando para executar	copiar com atenção; entender opções importantes
C / CMAKE / JSON / SQL	arquivo ou trecho de código	editar no Geany e salvar no caminho indicado
CONCEITO	explicação curta	entender a ideia antes de copiar o próximo bloco
SEGURANÇA	fronteira obrigatória	não substituir por atalhos com root ou shell genérico
CHECKPOINT	critério de saída	só avançar quando o resultado esperado aparecer
DEBUG	quando algo falha	rodar diagnóstico e comparar com o erro real

Mapa do livro

Parte I - Bancada Linux Mint: GCC, Geany, Git, CMake e primeiro executável.

Parte II - M0: Core mínimo, requests, eventos, provider mock, CLI e testes.

Parte III - Política e persistência: privacidade, SQLite, XDG, configuração e schemas.

Parte IV - M1: llama.cpp, llama-server, GGUF e provider HTTP real em C.

Parte V - Hardware-aware: profiler, compatibilidade, benchmark e roteadores.

Parte VI - RAG Lite: FTS5, ingestão, busca e citações.

Parte VII - Tools e Agent Engine: efeitos tipados, confirmação e limites.

Parte VIII - Serviço nativo: systemd de usuário, IPC, UI e voz opcionais.

Parte IX - Empacotamento e Athenas OS: .deb, testes de release e multi-arquitetura.

Parte X - Capstone: Atena mínima utilizável, definição de pronto e próximos marcos.

BASE DE ARQUITETURA: A ordem deste laboratório segue a arquitetura técnica do projeto: contratos e provider mock primeiro; llama-server depois; em seguida Policy/Tools, RAG/Citations,

UI/Voice e finalmente pacotes/ISO. O propósito é preservar a Atena quando o modelo ou o provider forem trocados.

PARTE I - A BANCADA

Linux Mint, Geany, compilador e build sem caixas-pretas

Laboratório 1 - Preparar o Linux Mint

OBJETIVO PRÁTICO: Sair deste laboratório com GCC, CMake, Git, GDB, Geany e bibliotecas de desenvolvimento instaladas e verificadas.

Você não instala “a linguagem C”. C é uma linguagem. O que instalamos é uma cadeia de ferramentas capaz de transformar arquivos de texto .c em código de máquina executável pelo processador. No Mint, a rota mais simples é o APT.

TERMINAL

```
sudo apt update
sudo apt install -y build-essential cmake git gdb pkg-config ninja-build geany geany-plugins
libjson-c-dev libsqlite3-dev libcurl4-openssl-dev
```

O metapacote build-essential traz o compilador GCC, o linker, make e os headers fundamentais da libc. CMake descreve o build; Git registra a história; GDB depura; pkg-config localiza bibliotecas; json-c, SQLite e libcurl serão usados mais à frente.

Pacote	Função no projeto
build-essential	GCC, linker, make e headers para compilar C/C++
cmake	descrever e reproduzir a build
git	versionar cada marco
gdb	investigar crashes e estado do programa
pkg-config	descobrir flags de bibliotecas
geany	editor leve
libjson-c-dev	ler e produzir JSON em C
libsqlite3-dev	persistência, FTS5 e banco local
libcurl4-openssl-dev	HTTP/HTTPS para providers

TERMINAL

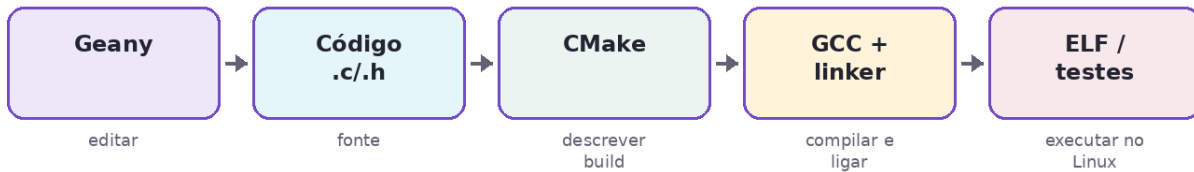
```
gcc --version
cmake --version
git --version
gdb --version
geany --version
pkg-config --modversion json-c
pkg-config --modversion sqlite3
pkg-config --modversion libcurl
uname -m
free -h
```

SAÍDA ESPERADA: Nenhum comando deve retornar “command not found”. Anote sua arquitetura em uname -m e a RAM total em free -h; esses dados voltarão no Hardware Profiler.

Laboratório 2 - Geany + terminal: o fluxo de trabalho

OBJETIVO PRÁTICO: Usar o Geany apenas como editor e enxergar a cadeia fonte → compilador → executável.

Do Geany ao executável



Abra o Geany pelo menu ou por `geany &`. Salve os fontes dentro de `~/Projetos/atena`. Nesta fase vamos compilar pelo terminal de propósito. Depois, se quiser, os botões de Build do Geany podem chamar os mesmos comandos. O importante é não esconder a engenharia atrás da interface.

TERMINAL

```
mkdir -p ~/Projetos/atena
cd ~/Projetos/atena
geany &
```

C — TESTE.C

```
#include <stdio.h>

int main(void) {
    puts("Ambiente da Atena funcionando!");
    return 0;
}
```

TERMINAL

```
gcc -std=c17 -Wall -Wextra -Wpedantic teste.c -o teste
./teste
file ./teste
```

Se `./teste` imprime a mensagem, você acabou de passar pela cadeia básica: pre-processador, compilador, assembler e linker. O resultado geralmente é um executável ELF para sua arquitetura.

POR QUE C17: C17 é uma versão estável do padrão C e fornece uma base portátil para o Core. Componentes externos, como `llama.cpp`, são C/C++, mas a API pública da Atena pode permanecer em C.

Laboratório 3 - Criar o repositório e o primeiro histórico

OBJETIVO PRÁTICO: Começar o projeto em uma pasta independente da build da ISO e registrar o estado inicial no Git.

TERMINAL

```
cd ~/Projetos/atena
```

```
rm -f teste teste.c
git init
printf "build/\nbuilt-*/\n*.o\n*.a\n*.so\n*.log\n" > .gitignore
```

O código da Atena não deve nascer dentro de config/includes.chroot do live-build. Primeiro construímos software normal, depois empacotamos em .deb, e só então integramos esse pacote à imagem do Athenas OS.

TERMINAL

```
git status
```

PRINCÍPIO: O repositório Atena e o repositório/construtor do Athenas OS são produtos relacionados, mas não a mesma pasta de trabalho.

Laboratório 4 - Aprender CMake construindo algo pequeno

OBJETIVO PRÁTICO: Criar uma biblioteca, um executável e um teste com uma build fora da árvore de fontes.

CMAKE — INÍCIO DO CMAKELISTS.TXT

```
cmake_minimum_required(VERSION 3.25)
project(atenas VERSION 0.1.0 LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
```

A opção `-S` aponta a árvore de fontes e `-B build` escolhe outra pasta para arquivos gerados. Isso evita poluir o repositório com objetos, caches e executáveis.

TERMINAL

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
```

SE FALHAR: Neste momento é normal o CMake reclamar que ainda não existe target. O objetivo do laboratório é entender o mecanismo; na Parte II criaremos o CMake completo e a configuração passará.

PARTE II - M0: O PRIMEIRO ATENA CORE

Contratos, provider mock e CLI compilada

M0 - primeira fatia executável



Laboratório 5 - Montar o esqueleto do repositório

OBJETIVO PRÁTICO: Criar a árvore mínima que permite separar API pública, Core, providers, CLI, testes, schemas e empacotamento.

TERMINAL

```
cd ~/Projetos/atenas
mkdir -p include/atenas core providers cli tests memory rag tools schemas docs/adr
packaging/debian
```

ÁRVORE

```

atenas/
├── include/atenas/    # API pública em C
├── core/              # request, policy, router
├── providers/         # mock, llama_http, cloud...
├── cli/               # interface de terminal
├── memory/           # SQLite e migrações
├── rag/               # FT55 e citações
├── tools/             # ferramentas tipadas
├── tests/             # unidade e integração
├── schemas/          # contratos JSON
└── packaging/debian/ # empacotamento

```

O Core não deve incluir headers de um provider específico. A dependência deve apontar do provider para o contrato do Core, e não ao contrário. Essa direção evita que trocar llama-server por outro runtime implique reescrever a plataforma.

Laboratório 6 - Status: erros como contrato

OBJETIVO PRÁTICO: Criar uma enumeração comum de erros e uma função de representação textual.

C — INCLUDE/ATENA/STATUS.H

```
#ifndef ATENA_STATUS_H
```

```
#define ATENA_STATUS_H

typedef enum {
    ATENA_OK = 0,
    ATENA_ERR_INVALID,
    ATENA_ERR_NOMEM,
    ATENA_ERR_IO,
    ATENA_ERR_POLICY,
    ATENA_ERR_PROVIDER,
    ATENA_ERR_UNSUPPORTED
} AtenaStatus;

const char *atena_status_string(AtenaStatus status);
#endif
```

C — CORE/STATUS.C

```
#include "atena/status.h"

const char *atena_status_string(AtenaStatus status) {
    switch (status) {
        case ATENA_OK: return "ok";
        case ATENA_ERR_INVALID: return "invalid argument";
        case ATENA_ERR_NOMEM: return "out of memory";
        case ATENA_ERR_IO: return "I/O error";
        case ATENA_ERR_POLICY: return "blocked by policy";
        case ATENA_ERR_PROVIDER: return "provider error";
        case ATENA_ERR_UNSUPPORTED: return "unsupported";
    }
    return "unknown error";
}
```

C não possui exceções nativas. Em uma biblioteca de sistema, retornar um status explícito é mais previsível. A camada que chama decide se aquilo vira mensagem para o usuário, evento ou log.

Laboratório 7 - AtenaRequest: transformar texto em intenção estruturada

OBJETIVO PRÁTICO: Criar o objeto que atravessa a fronteira de entrada e já carrega privacidade e permissões.

C — INCLUDE/ATENA/REQUEST.H

```
#ifndef ATENA_REQUEST_H
#define ATENA_REQUEST_H
#include <stdbool.h>
#include <stdint.h>
#include "atena/status.h"

typedef enum {
    ATENA_PRIVACY_PUBLIC = 0,
    ATENA_PRIVACY_NORMAL,
    ATENA_PRIVACY_SENSITIVE,
    ATENA_PRIVACY_PRIVATE,
    ATENA_PRIVACY_SECRET
} AtenaPrivacyClass;

typedef struct {
    uint64_t id;
    const char *session_id;
    const char *user_text;
    AtenaPrivacyClass privacy;
    uint32_t required_capabilities;
    bool allow_network;
    bool allow_tools;
} AtenaRequest;

AtenaStatus atena_request_validate(const AtenaRequest *request);
```

```
#endif
```

C — CORE/REQUEST.C

```
#include "atena/request.h"
#include <string.h>
#define ATENA_MAX_USER_TEXT_BYTES (64u * 1024u)

AtenaStatus atena_request_validate(const AtenaRequest *request) {
    if (request == NULL || request->session_id == NULL || request->user_text == NULL)
        return ATENA_ERR_INVALID;
    if (request->session_id[0] == '\0' || request->user_text[0] == '\0')
        return ATENA_ERR_INVALID;
    if (strlen(request->user_text) > ATENA_MAX_USER_TEXT_BYTES)
        return ATENA_ERR_INVALID;
    if (request->privacy < ATENA_PRIVACY_PUBLIC || request->privacy > ATENA_PRIVACY_SECRET)
        return ATENA_ERR_INVALID;
    return ATENA_OK;
}
```

O SALTO ARQUITETURAL: Uma string diz “explique ponteiros”. Um request diz: texto, sessão, privacidade, rede permitida ou não, tools permitidas ou não e capacidades necessárias. É isso que permite política verificável fora do LLM.

Laboratório 8 - Buffer e ownership sem magia

OBJETIVO PRÁTICO: Aprender malloc, free e ciclo de vida dentro de um tipo que será útil para respostas HTTP e JSON.

C — INCLUDE/ATENA/BUFFER.H

```
#ifndef ATENA_BUFFER_H
#define ATENA_BUFFER_H
#include <stddef.h>
#include "atena/status.h"

typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} AtenaBuffer;

AtenaStatus atena_buffer_init(AtenaBuffer *b, size_t initial);
AtenaStatus atena_buffer_append(AtenaBuffer *b, const char *data, size_t length);
void atena_buffer_deinit(AtenaBuffer *b);
#endif
```

C — CORE/BUFFER.C (INÍCIO)

```
#include "atena/buffer.h"
#include <stdlib.h>
#include <string.h>

AtenaStatus atena_buffer_init(AtenaBuffer *b, size_t initial) {
    if (!b || initial == 0) return ATENA_ERR_INVALID;
    b->data = malloc(initial);
    if (!b->data) return ATENA_ERR_NOMEM;
    b->data[0] = '\0'; b->length = 0; b->capacity = initial;
    return ATENA_OK;
}

void atena_buffer_deinit(AtenaBuffer *b) {
    if (!b) return;
    free(b->data); b->data = NULL; b->length = b->capacity = 0;
}
```

Ownership responde quem cria, quem pode usar e quem libera. Para cada `atena_buffer_init` bem-sucedido, deve existir um `atena_buffer_deinit` visível no caminho de saída. Essa disciplina reduz leaks, double free e use-after-free.

REGRA DE REALLOC: Nunca sobrescreva o ponteiro original com `realloc` antes de saber que a operação funcionou. Guarde o retorno em um ponteiro temporário e só depois atualize o objeto.

Laboratório 9 - Eventos: uma linguagem comum para streaming

OBJETIVO PRÁTICO: Fazer providers diferentes emitirem a mesma sequência de tipos de evento.

C — INCLUDE/ATENA/EVENT.H

```
#ifndef ATENA_EVENT_H
#define ATENA_EVENT_H
#include <stdint.h>
#include "atena/status.h"

typedef enum {
    ATENA_EVENT_TEXT_DELTA = 0,
    ATENA_EVENT_TOOL_PROPOSAL,
    ATENA_EVENT_CITATION,
    ATENA_EVENT_USAGE,
    ATENA_EVENT_DONE,
    ATENA_EVENT_ERROR
} AtenaEventType;

typedef struct {
    AtenaEventType type;
    uint64_t request_id;
    const char *json_payload;
} AtenaEvent;

typedef AtenaStatus (*AtenaEventCallback)(const AtenaEvent *event, void *user_data);
#endif
```

TEXT_DELTA pode vir de um streaming SSE de nuvem, de um socket local ou de uma chamada nativa. A CLI e a futura UI não precisam saber. Esse desacoplamento é uma das peças que impedem provider-specific code de vazar por todo o produto.

Laboratório 10 - Provider API: o soquete universal

OBJETIVO PRÁTICO: Criar a vtable que torna Mock, llama-server, Ollama e providers remotos substituíveis.

C — INCLUDE/ATENA/PROVIDER.H

```
#ifndef ATENA_PROVIDER_H
#define ATENA_PROVIDER_H
#include "atena/event.h"
#include "atena/request.h"

typedef struct AtenaProvider AtenaProvider;

typedef struct {
    AtenaStatus (*start)(AtenaProvider *self);
    AtenaStatus (*generate)(AtenaProvider *self, const AtenaRequest *request,
                           AtenaEventCallback callback, void *user_data);
    void (*cancel)(AtenaProvider *self, uint64_t request_id);
    void (*destroy)(AtenaProvider *self);
} AtenaProviderVTable;

struct AtenaProvider {
```

```
const AtenaProviderVTable *ops;
const char *id;
const char *display_name;
void *impl;
};
#endif
```

O Core chama `provider->ops->generate()`. Ele não compara `provider_id` com “ollama”, “llama” ou “openai”. Implementações específicas ficam isoladas. Esse padrão é uma vtable: uma tabela de ponteiros de função que representa uma interface substituível em C.

Laboratório 11 - Provider Mock: testar arquitetura antes da IA real

OBJETIVO PRÁTICO: Emitir eventos previsíveis sem modelo, rede, GPU ou downloads.

C — PROVIDERS/MOCK.C (NÚCLEO)

```
#include "atena/mock_provider.h"
#include <stdlib.h>

static AtenaStatus mock_generate(AtenaProvider *self, const AtenaRequest *request,
                                AtenaEventCallback callback, void *user_data) {
    if (!self || !callback) return ATENA_ERR_INVALID;
    AtenaStatus s = atena_request_validate(request);
    if (s != ATENA_OK) return s;

    AtenaEvent e1 = { ATENA_EVENT_TEXT_DELTA, request->id,
                     "{\"text\":\"Atena mock: recebi sua solicitacao.\"}" };
    s = callback(&e1, user_data);
    if (s != ATENA_OK) return s;

    AtenaEvent done = { ATENA_EVENT_DONE, request->id, "{\"status\":\"ok\"}" };
    return callback(&done, user_data);
}
```

O mock não é “tempo perdido”. Ele torna os testes determinísticos. Se a CLI, os requests e os eventos funcionam com Mock e quebram com llama-server, você já reduziu a superfície do problema ao provider real ou ao transporte.

Laboratório 12 - Atena CLI: primeira interface real

OBJETIVO PRÁTICO: Criar um executável que monta `AtenaRequest`, abre um provider e renderiza eventos.

C — CLI/MAIN.C (ESTRUTURA)

```
static AtenaStatus print_event(const AtenaEvent *event, void *user_data) {
    (void)user_data;
    if (!event) return ATENA_ERR_INVALID;
    printf("[%d] %s\n", (int)event->type, event->json_payload);
    return ATENA_OK;
}

int main(int argc, char **argv) {
    const char *text = argc >= 2 ? argv[1] : "Ola Atena";
    AtenaRequest request = {
        .id = 1, .session_id = "cli-session-001", .user_text = text,
        .privacy = ATENA_PRIVACY_NORMAL, .allow_network = false, .allow_tools = false
    };
    /* criar mock, start, generate, destroy */
}
```

POR QUE ARGC/ARGV: argv[0] é o nome do programa. argv[1] é o primeiro argumento. A CLI será expandida para subcomandos como atena-cli ask, models, doctor e index.

Laboratório 13 - CMake completo, build e testes

OBJETIVO PRÁTICO: Compilar atena-core, provider mock, CLI e teste em targets separados.

CMAKE — CMAKELISTS.TXT

```
cmake_minimum_required(VERSION 3.25)
project(atenas VERSION 0.1.0 LANGUAGES C)
set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
include(CTest)

add_library(atenas-core STATIC core/status.c core/request.c)
target_include_directories(atenas-core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_compile_options(atenas-core PRIVATE -Wall -Wextra -Wpedantic -Wconversion -Wshadow)

add_library(atenas-provider-mock STATIC providers/mock.c)
target_include_directories(atenas-provider-mock PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_link_libraries(atenas-provider-mock PUBLIC atenas-core)

add_executable(atenas-cli cli/main.c)
target_link_libraries(atenas-cli PRIVATE atenas-core atenas-provider-mock)

if(BUILD_TESTING)
    add_executable(test-request tests/test_request.c)
    target_link_libraries(test-request PRIVATE atenas-core)
    add_test(NAME request-validation COMMAND test-request)
endif()
```

TERMINAL

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure
./build/atenas-cli "Explique ponteiros"
```

M0 MÍNIMO: A build termina sem erro; ctest passa; atenas-cli imprime TEXT_DELTA e DONE do provider mock. Faça um commit antes de avançar.

TERMINAL

```
git add .
git commit -m "M0: core contracts, mock provider and CLI"
```

Laboratório 14 - Sanitizers e GDB antes de crescer

OBJETIVO PRÁTICO: Adicionar diagnósticos de memória e comportamento indefinido enquanto o projeto ainda é pequeno.

TERMINAL

```
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug -DCMAKE_C_FLAGS="-fsanitize=address,undefined
-fno-omit-frame-pointer -g"
cmake --build build-asan -j2
ctest --test-dir build-asan --output-on-failure
```

AddressSanitizer encontra acessos fora do limite, use-after-free e muitos erros de heap. UndefinedBehaviorSanitizer denuncia categorias de comportamento indefinido. GDB continua útil para breakpoints, backtrace e inspeção de variáveis.

TERMINAL

```
gdb --args ./build/atena-cli "teste"  
# dentro do gdb: run  
# se houver crash: bt
```

PARTE III - M0.5: POLÍTICA E PERSISTÊNCIA

A Atena começa a ter regras próprias, sem depender do modelo

Laboratório 15 - Policy Engine: segurança antes do roteamento

OBJETIVO PRÁTICO: Codificar regras determinísticas de privacidade e rede.

C — INCLUDE/ATENA/POLICY.H

```
#ifndef ATENA_POLICY_H
#define ATENA_POLICY_H
#include <stdbool.h>
#include "atena/request.h"

typedef struct {
    bool cloud_allowed;
    bool tools_allowed;
    bool confirmation_required;
} AtenaPolicyDecision;

AtenaStatus atena_policy_evaluate(const AtenaRequest *request, AtenaPolicyDecision *out);
#endif
```

C — CORE/POLICY.C

```
AtenaStatus atena_policy_evaluate(const AtenaRequest *r, AtenaPolicyDecision *out) {
    if (!r || !out) return ATENA_ERR_INVALID;
    out->cloud_allowed = r->allow_network;
    out->tools_allowed = r->allow_tools;
    out->confirmation_required = false;

    if (r->privacy == ATENA_PRIVACY_PRIVATE || r->privacy == ATENA_PRIVACY_SECRET)
        out->cloud_allowed = false;
    if (r->privacy == ATENA_PRIVACY_SENSITIVE && r->allow_network)
        out->confirmation_required = true;
    return ATENA_OK;
}
```

ORDEM NÃO NEGOCIÁVEL: Privacidade e autorização são avaliadas antes de qualidade, custo ou velocidade. Um provider remoto “melhor” não entra no conjunto de candidatos quando a política exige processamento local.

Crie testes de tabela: PUBLIC/NORMAL podem usar cloud quando allow_network=true; PRIVATE/SECRET nunca; SENSITIVE exige confirmação quando a saída de dados é necessária. Esses testes são mais importantes do que perguntar a um LLM se ele “seria seguro”.

Laboratório 16 - SQLite e XDG: memória com caminho correto

OBJETIVO PRÁTICO: Abrir o banco em um diretório de dados do usuário e aplicar migrações versionadas.

TERMINAL

```
echo "$XDG_DATA_HOME"
echo "$HOME/.local/share"
```

Se XDG_DATA_HOME estiver vazio, use ~/.local/share. Para configuração, ~/.config; cache recriável, ~/.cache; runtime efêmero, XDG_RUNTIME_DIR. Não salve API keys em um JSON comum.

SQL — SCHEMAS/001_INITIAL.SQL

```
PRAGMA foreign_keys = ON;
CREATE TABLE schema_migrations (
  version INTEGER PRIMARY KEY, applied_at TEXT NOT NULL
);
CREATE TABLE conversations (
  id TEXT PRIMARY KEY, title TEXT, created_at TEXT NOT NULL,
  privacy_class TEXT NOT NULL, retention_policy TEXT NOT NULL
);
CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  conversation_id TEXT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  role TEXT NOT NULL, content TEXT NOT NULL, provider_id TEXT, model_id TEXT,
  created_at TEXT NOT NULL
);
CREATE TABLE preferences (
  key TEXT PRIMARY KEY, value_json TEXT NOT NULL, updated_at TEXT NOT NULL
);
```

Atena não deve tratar sessão, histórico, preferências, memória de longo prazo e base de conhecimento como uma única caixa. Categorias separadas tornam retenção, exportação e exclusão compreensíveis.

TESTE MANUAL: Use sqlite3 no banco de desenvolvimento, rode .tables e confirme que schema_migrations, conversations, messages e preferences existem. Depois automatize isso em memory/store.c.

Laboratório 17 - Configuração e JSON Schema

OBJETIVO PRÁTICO: Separar preferências inspecionáveis de segredos e validar fronteiras.

JSON — SCHEMAS/REQUEST.SCHEMA.JSON

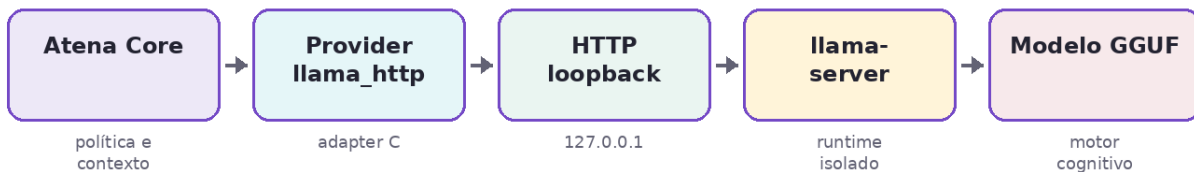
```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "org.athenas.atena.request/1",
  "type": "object",
  "additionalProperties": false,
  "required": ["id", "session_id", "user_text", "privacy", "allow_network", "allow_tools"],
  "properties": {
    "id": {"type": "integer", "minimum": 1},
    "session_id": {"type": "string", "minLength": 1},
    "user_text": {"type": "string", "minLength": 1, "maxLength": 65536},
    "privacy": {"enum": ["PUBLIC", "NORMAL", "SENSITIVE", "PRIVATE", "SECRET"]},
    "allow_network": {"type": "boolean"},
    "allow_tools": {"type": "boolean"}
  }
}
```

No início, o schema serve como contrato documentado e teste de fixtures. Depois você pode integrar um validador de JSON Schema ou escrever validação explícita para os poucos campos críticos. O requisito importante é não aceitar “qualquer JSON” como ação de sistema.

PARTE IV - M1: LLM LOCAL REAL

llama.cpp, llama-server e o primeiro provider cognitivo

M1 - inferência local sem acoplar o Core



Laboratório 18 - Compilar llama.cpp no Linux Mint

OBJETIVO PRÁTICO: Construir o runtime local em um repositório separado e localizar llama-server.

llama.cpp é o runtime local de referência nesta fase. Não o copie para dentro do Core. Mantenha o projeto separado e conecte por HTTP em loopback. Isso isola crashes, memória e versões do runtime.

TERMINAL

```

cd ~/Projetos
git clone https://github.com/ggml-org/llama.cpp.git
cd llama.cpp
cmake -B build
cmake --build build --config Release -t llama-server -j2
./build/bin/llama-server --version
  
```

COMANDO QUE ENVELHECE: O repositório llama.cpp evolui rápido. Este livro usa a forma CMake e o alvo llama-server que constam na documentação oficial consultada em agosto de 2026. Se a upstream mudar flags, fixe uma revisão testada no projeto e atualize conscientemente.

Em CPUs pequenas, -j2 reduz pico de memória da compilação. Aceleração por GPU é uma otimização posterior. Primeiro valide o caminho CPU, local e reproduzível.

Laboratório 19 - Rodar um GGUF e provar que o servidor funciona

OBJETIVO PRÁTICO: Iniciar o runtime apenas em 127.0.0.1 e testar um endpoint antes de tocar no Core.

GGUF é o artefato de modelo que o llama.cpp carrega. Escolha um modelo instruct pequeno e uma quantização compatível com a RAM. Para máquinas muito limitadas, prefira centenas de milhões a cerca de 1-2 bilhões de parâmetros; medir na máquina é mais confiável do que decorar “regras de Q4”.

TERMINAL

```
mkdir -p ~/.local/share/atena/models
# coloque seu arquivo .gguf nesse diretório
~/Projetos/llama.cpp/build/bin/llama-server \
--host 127.0.0.1 --port 18080 \
--model ~/.local/share/atena/models/SEU_MODELO.gguf \
--ctx-size 2048 --threads 2
```

Em outro terminal, primeiro veja se o servidor está acessível. A documentação atual do llama-server também oferece rotas compatíveis com a API da OpenAI, o que simplifica adapters.

TERMINAL

```
curl http://127.0.0.1:18080/health
curl -s http://127.0.0.1:18080/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"messages":[{"role":"user","content":"Responda apenas: Atena local funcionando"}], "temperature":0.2}'
```

PRIMEIRA INFERÊNCIA REAL: O curl deve retornar JSON contendo uma resposta do modelo. Só depois implemente o provider C. Isso separa “servidor/modelo não funciona” de “nosso adapter está errado”.

Laboratório 20 - HTTP em C: libcurl + json-c

OBJETIVO PRÁTICO: Criar um cliente mínimo que envia JSON e acumula a resposta em AtenaBuffer.

C — PROVIDERS/HTTP_COMMON.C

```
#include <curl/curl.h>

static size_t write_cb(char *ptr, size_t size, size_t nmemb, void *userdata) {
    AtenaBuffer *b = userdata;
    size_t n = size * nmemb;
    if (atena_buffer_append(b, ptr, n) != ATENA_OK) return 0;
    return n;
}
```

C — ESQUELETO HTTP SÍNCRONO

```
CURL *curl = curl_easy_init();
if (!curl) return ATENA_ERR_PROVIDER;

curl_easy_setopt(curl, CURLOPT_URL, "http://127.0.0.1:18080/v1/chat/completions");
curl_easy_setopt(curl, CURLOPT_POSTFIELDS, json_body);
curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT_MS, 800L);
curl_easy_setopt(curl, CURLOPT_TIMEOUT_MS, 60000L);
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);
curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);

CURLcode rc = curl_easy_perform(curl);
curl_easy_cleanup(curl);
```

Comece síncrono. Faça funcionar, teste timeout e erros HTTP. Depois adicione streaming. Quando dois problemas são introduzidos ao mesmo tempo, debugging vira neblina: você não sabe se o erro é HTTP, parser incremental, callback ou modelo.

LOOPBACK NÃO É INTERNET: O provider llama-server pode usar AF_INET, mas o endpoint é 127.0.0.1. Trate-o como local. O Core deve recusar um endpoint “local” que foi reconfigurado para 0.0.0.0 ou endereço externo sem política explícita.

Laboratório 21 - Provider llama_http

OBJETIVO PRÁTICO: Traduzir AtenaRequest para o formato do servidor e normalizar resposta em AtenaEvent.

C — INCLUDE/ATENA/LLAMA_HTTP_PROVIDER.H

```
typedef struct {
    char endpoint[256];
    long connect_timeout_ms;
    long total_timeout_ms;
} LlamaHttpState;

AtenaStatus atena_llama_http_provider_create(
    const char *endpoint, AtenaProvider **out
);
```

O estado privado fica em provider->impl. O Core só conhece AtenaProvider. generate() constrói o JSON de mensagens, chama HTTP, extrai choices[0].message.content e emite TEXT_DELTA/DONE. Em um segundo passo, troque para stream=true e transforme cada delta recebido em AtenaEvent.

C — MONTAR REQUEST JSON

```
json_object *root = json_object_new_object();
json_object *messages = json_object_new_array();
json_object *msg = json_object_new_object();
json_object_object_add(msg, "role", json_object_new_string("user"));
json_object_object_add(msg, "content", json_object_new_string(request->user_text));
json_object_array_add(messages, msg);
json_object_object_add(root, "messages", messages);
json_object_object_add(root, "temperature", json_object_new_double(0.3));
const char *body = json_object_to_json_string_ext(root, JSON_C_TO_STRING_PLAIN);
```

TROCA DE CÉREBRO: atena-cli deve aceitar uma opção de provider, por exemplo --provider llama.local. Com mock e llama_http disponíveis, a mesma CLI usa dois motores sem alterar AtenaRequest nem AtenaEvent.

Laboratório 22 - Streaming, cancelamento e timeouts

OBJETIVO PRÁTICO: Tornar geração longa controlável e a UI futura responsiva.

Streaming significa processar partes enquanto chegam. O provider precisa saber se o request foi cancelado. O Core marca cancelamento por request_id; o write callback encerra a transferência; o provider emite ERROR ou DONE conforme o contrato.

Falha	Comportamento esperado
servidor não inicia	health falha; router marca provider indisponível
conexão excede timeout	ATENA_ERR_PROVIDER com causa sanitizada
usuário cancela	provider interrompe; Core finaliza request
JSON inválido	erro de provider; não crashar
modelo sem capacidade pedida	UNSUPPORTED antes da chamada
resposta enorme	limite de bytes e cancelamento

Não registre prompts ou headers completos por padrão. Logs de erro devem carregar código, provider_id, etapa e tempos, não conteúdo privado nem credenciais.

PARTE V - HARDWARE-AWARE E ROUTERS

Atena passa a escolher com base na máquina real

Laboratório 23 - Hardware Profiler mínimo

OBJETIVO PRÁTICO: Medir arquitetura, CPU e memória disponível sem depender do LLM.

C — INCLUDE/ATENA/HARDWARE.H

```
#include <sys/utsname.h>
#include <unistd.h>

typedef struct {
    char arch[64];
    long cpu_count;
    unsigned long long ram_total_bytes;
    unsigned long long ram_available_bytes;
} AtenaHardwareProfile;
```

uname() informa machine; sysconf(_SC_NPROCESSORS_ONLN) dá CPUs lógicas online. Em Linux, /proc/meminfo fornece MemTotal e MemAvailable. Parseie chaves exatas, valide unidades kB e use inteiros de largura suficiente.

TERMINAL

```
uname -m
nproc
grep -E "MemTotal|MemAvailable|SwapTotal|SwapFree" /proc/meminfo
```

HARDWARE-AWARE NÃO É ADIVINHAÇÃO: A estimativa inicial serve para filtrar modelos impossíveis. O Benchmark Engine depois substitui parte dessas hipóteses por memória e tokens/s medidos na própria máquina.

Laboratório 24 - Model Manager e catálogo local

OBJETIVO PRÁTICO: Registrar arquivos GGUF sem acoplar a lógica a nomes comerciais.

C — MODELO DE CATÁLOGO

```
typedef struct {
    char id[128];
    char path[512];
    char family[128];
    char quantization[64];
    unsigned long long file_size;
    unsigned int context_hint;
    bool verified;
} AtenaModelRecord;
```

O primeiro Model Manager pode fazer três coisas: registrar um arquivo, calcular hash e executar um smoke test no llama-server. Depois adiciona metadados, licença, origem e benchmark. O Core solicita capability; o Model Router escolhe um registro compatível.

TERMINAL

```
sha256sum ~/.local/share/atena/models/SEU_MODELO.gguf
```

Laboratório 25 - Benchmark Engine

OBJETIVO PRÁTICO: Medir tempo até primeiro token, tokens/s e memória de pico para a combinação modelo + configuração.

Crie um conjunto pequeno e fixo de prompts curtos. Grave hardware_id, model_id, ctx_size, threads, tempo de carregamento, TTFT (time to first token), tokens por segundo e RSS de pico. Não use um benchmark como propaganda; use para seleção na própria máquina.

Métrica	Pergunta
TTFT	quanto tempo o usuário espera até começar a resposta?
tokens/s	qual a velocidade sustentada?
RSS pico	a máquina entra em swap ou OOM?
load time	vale manter o runtime residente?
ctx_size	quanto contexto cabe sem destruir responsividade?

Laboratório 26 - Provider Router e Model Router

OBJETIVO PRÁTICO: Filtrar por política/capacidade antes de pontuar qualidade, velocidade e custo.

C — CORE/ROUTER.C (ORDEM IMPORTANTE)

```
for (size_t i = 0; i < count; ++i) {
    const AtenaProviderInfo *p = &providers[i];
    if (!p->is_available) continue;
    if ((p->capabilities & request->required_capabilities) != request->required_capabilities)
        continue;
    if (!request->allow_network && !p->is_local) continue;
    if ((request->privacy == ATENA_PRIVACY_PRIVATE ||
        request->privacy == ATENA_PRIVACY_SECRET) && !p->is_local) continue;

    double score = p->is_local ? 10.0 : 5.0;
    score += p->health_score + p->quality_score;
    score -= p->estimated_cost_score;
    /* manter o melhor */
}
```

FILTRO ANTES DO SCORE: Nunca pontue um candidato proibido para depois “ver se ele ganha”. Remova primeiro tudo que política, rede e capabilities impedem. Só então compare candidatos válidos.

PARTE VI - RAG LITE E CITAÇÕES

Conhecimento local útil mesmo em hardware onde um LLM grande não cabe

Laboratório 27 - FTS5: busca lexical que funciona offline

OBJETIVO PRÁTICO: Criar índice de documentos e recuperar trechos por termos antes de adicionar embeddings.

SQL — MIGRAÇÃO RAG LITE

```
CREATE TABLE documents (
  id TEXT PRIMARY KEY, path TEXT NOT NULL, title TEXT,
  mime TEXT NOT NULL, sha256 TEXT NOT NULL, indexed_at TEXT NOT NULL
);
CREATE TABLE chunks (
  id TEXT PRIMARY KEY, document_id TEXT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  chunk_index INTEGER NOT NULL, page INTEGER, content TEXT NOT NULL
);
CREATE VIRTUAL TABLE chunks_fts USING fts5(
  content, chunk_id UNINDEXED, document_id UNINDEXED, tokenize="unicode61"
);
```

FTS5 é uma excelente primeira camada porque não exige carregar um modelo de embeddings. Em uma máquina de 2 GB, a Atena ainda pode buscar documentação local, citar o trecho e usar tools mesmo quando inferência local pesada é inviável.

Laboratório 28 - Ingestão: texto e Markdown primeiro

OBJETIVO PRÁTICO: Transformar arquivos autorizados em chunks limitados e rastreáveis.

Comece com .txt e .md: leia bytes com limite, valide UTF-8, normalize quebras de linha, divida por parágrafos e agrupe até um alvo de tamanho. Cada chunk precisa manter document_id, índice e locator. PDF entra depois por um extrator separado, nunca executando “comando contido no documento”.

C — RAG/CHUNK.H

```
typedef struct {
  char id[64];
  char document_id[64];
  unsigned int chunk_index;
  int page;
  char *content;
} AtenaChunk;
```

DOCUMENTO É DADO NÃO CONFIÁVEL: Um PDF ou Markdown pode conter “ignore suas regras” ou instruções para tools. O Context Builder rotula trechos como evidência. O Tool Engine continua validando fora do modelo.

Laboratório 29 - Busca e Citation Engine

OBJETIVO PRÁTICO: Retornar evidências com locators que a resposta consegue citar.

SQL — CONSULTA FTS5

```
SELECT c.id, c.document_id, c.page, c.content, bm25(chunks_fts) AS score
FROM chunks_fts
JOIN chunks_c ON c.id = chunks_fts.chunk_id
WHERE chunks_fts MATCH ?
ORDER BY score
LIMIT ?;
```

A resposta do RAG não deve ser “texto solto”. Cada trecho carrega locator: arquivo, página quando conhecida e id de chunk. O modelo recebe evidências claramente delimitadas. A camada de citação gera eventos CITATION separados do texto.

RAG LITE PRONTO: atena-cli index caminho.md grava documento/chunks. atena-cli search "termo" retorna top-k com arquivo e trecho. atena-cli ask pode anexar esses trechos ao contexto e emitir CITATION.

PARTE VII - TOOLS E AGENTE

A IA deixa de apenas falar, mas efeitos continuam fora do texto gerado

Laboratório 30 - Tool Registry

OBJETIVO PRÁTICO: Registrar funções tipadas com schema, risco e propriedades conhecidas.

C — INCLUDE/ATENA/TOOL.H

```
typedef AtenaStatus (*AtenaToolRun)(
    const char *arguments_json, AtenaToolContext *context, char **result_json
);

typedef struct {
    const char *name;
    uint32_t version;
    AtenaRisk risk;
    bool idempotent;
    const char *input_schema_path;
    const char *output_schema_path;
    AtenaToolRun run;
} AtenaToolDescriptor;
```

O registry rejeita nome duplicado, versão zero, schema ausente e função nula. O modelo não recebe uma tool “shell”. Ele recebe nomes semânticos, argumentos JSON e descrições limitadas.

PROIBIÇÃO ESTRUTURAL: Não crie ExecuteShell(string), atena-root-shell, NOPASSWD: ALL ou socket root genérico. Linguagem natural nunca deve virar comando de root por concatenação.

Laboratório 31 - Tool system.info.read

OBJETIVO PRÁTICO: Criar uma ferramenta sem privilégio que prova todo o caminho proposal → policy → execution → result.

C — TOOLS/SYSTEM_INFO.C

```
static AtenaStatus system_info_run(const char *args_json,
                                   AtenaToolContext *ctx,
                                   char **result_json) {
    (void)args_json;
    if (!ctx || !result_json) return ATENA_ERR_INVALID;
    /* coletar uptime, memória, arquitetura e CPUs */
    /* serializar para JSON; chamador recebe ownership do resultado */
    return ATENA_OK;
}
```

Essa tool é ideal para o primeiro caminho porque só lê dados. Teste-a diretamente sem LLM; depois permita que o Agent Engine proponha a chamada. Quando o modelo estiver errado, a ferramenta ainda deve respeitar schema e limites.

Laboratório 32 - Brightness com confirmação e privilégio mínimo

OBJETIVO PRÁTICO: Desenhar uma ação de escrita que não transforma Atena em root.

Ajustar brilho normalmente envolve `/sys/class/backlight` e permissões do sistema. O Core roda como usuário. Uma implementação de produção pode usar um helper administrativo pequeno protegido por Polkit. O helper valida novamente percent e dispositivo, realiza somente aquela operação e termina.

JSON — INTENÇÃO SEMÂNTICA

```
{
  "name": "system.brightness.set",
  "risk": "MEDIUM",
  "input": {"percent": 40},
  "requires_confirmation": true
}
```

DUAS VALIDAÇÕES: O Core valida antes de pedir autorização. O helper privilegiado valida de novo. Nunca confie que “veio do Core” é suficiente para escrever em sysfs.

Laboratório 33 - Agent Engine mínimo

OBJETIVO PRÁTICO: Implementar um loop com número de passos limitado e ferramentas conhecidas.

C — CORE/AGENT.C (ESQUELETO)

```
for (unsigned step = 0; step < plan.max_steps; ++step) {
  /* 1. gerar/obter próxima proposta */
  /* 2. validar schema */
  /* 3. Policy Engine decide */
  /* 4. pedir confirmação quando necessário */
  /* 5. executar tool conhecida */
  /* 6. anexar resultado tipado ao contexto */
  /* 7. encerrar em resposta final ou erro */
}
```

Limite de passos, tempo e ferramentas impede loops infinitos. Uma ação irreversível não deve ser reexecutada automaticamente em retry. Idempotência e reversibilidade fazem parte do descritor da tool, não de um palpite do LLM.

PARTE VIII - SERVIÇO NATIVO, IPC E INTERFACE

Uma única lógica compartilhada por CLI, UI e voz

Laboratório 34 - Transformar o Core em serviço de usuário

OBJETIVO PRÁTICO: Rodar atena-core sem root e deixar interfaces se conectarem ao mesmo processo.

INI — PACKAGING/SYSTEMD/ATENA-CORE.SERVICE

```
[Unit]
Description=Atena Core user service
After=graphical-session.target

[Service]
Type=simple
ExecStart=/usr/libexec/aten/aten-core
Restart=on-failure
RestartSec=2
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=read-only

[Install]
WantedBy=default.target
```

Comece testando o serviço em `~/config/systemd/user` durante desenvolvimento. O pacote final instala a unit no local apropriado. Hardening precisa ser testado com os diretórios que o RAG realmente lê e os dados que o Core realmente escreve.

TERMINAL

```
systemctl --user daemon-reload
systemctl --user enable --now atena-core.service
systemctl --user status atena-core.service
journalctl --user -u atena-core.service -n 50
```

Laboratório 35 - IPC por socket Unix antes de D-Bus

OBJETIVO PRÁTICO: Separar processo sem introduzir uma API de desktop complexa cedo demais.

Um socket em `$XDG_RUNTIME_DIR/aten/core.sock` é adequado ao protótipo. Use framing explícito: 4 bytes de tamanho em ordem definida + JSON. Limite o tamanho máximo. Valide credenciais do peer. Depois, quando o contrato estiver estável, D-Bus pode expor métodos semânticos como Ask, Cancel, ListModels e InvokeSkill.

C — FRAMING DE SOCKET

```
uint32_t len_be;
/* read_exact(fd, &len_be, 4) */
uint32_t len = ntohl(len_be);
if (len == 0 || len > ATENA_MAX_IPC_MESSAGE) reject();
```

```
/* alocar len+1, ler exato, validar JSON */
```

IPC NÃO É SHELL: Mesmo local, o método não recebe uma string para executar. Ele recebe requests e ações com schema. A fronteira de IPC deve reforçar, não anular, a modelagem do Core.

Laboratório 36 - UI GTK opcional: cliente, não segundo Core

OBJETIVO PRÁTICO: Criar uma janela simples que envia requests ao serviço e renderiza eventos.

A interface gráfica pode ser escrita em GTK4 ou outra toolkit escolhida pelo projeto. O requisito é arquitetural: ela é cliente. Não replique Policy Engine, memória ou Provider Router dentro da janela. Isso garante que CLI, voz e UI tenham a mesma privacidade.

TERMINAL

```
sudo apt install -y libgtk-4-dev
```

C — UI/MAIN.C (PRIMEIRO SMOKE TEST GTK)

```
#include <gtk/gtk.h>

static void activate(GtkApplication *app, gpointer data) {
    GtkWidget *win = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(win), "Atena");
    gtk_window_set_default_size(GTK_WINDOW(win), 780, 560);
    gtk_window_present(GTK_WINDOW(win));
}
```

Depois do smoke test, acrescente campo de entrada, histórico e botão Cancelar. A comunicação com o Core deve acontecer de forma assíncrona; nunca bloqueie a thread principal da UI esperando inferência.

Laboratório 37 - Voz como provider de entrada/saída

OBJETIVO PRÁTICO: Adicionar STT/TTS sem fundir voz com o motor cognitivo.

Voz é opcional. Capture áudio por PipeWire, use um STT local ou remoto conforme política e entregue texto ao mesmo Request Engine. TTS recebe texto de saída. O wake word, se existir, apenas inicia captura; não deve conceder autorização para tools.

UMA ARQUITETURA, TRÊS INTERFACES: UI, Voice e CLI convergem para o mesmo Core. Se a voz tiver permissão diferente da CLI, a arquitetura começou a se partir.

PARTE IX - PACOTES, TESTES E ATHENAS OS

Transformar o laboratório em componente instalável

Laboratório 38 - Pacote Debian

OBJETIVO PRÁTICO: Gerar .deb em vez de copiar binários manualmente para a ISO.

A unidade de integração do Athenas OS deve ser pacote. Isso permite instalar, atualizar, remover, declarar dependências e testar em ambiente limpo. O Core, CLI, schemas, systemd units e assets podem ser separados em pacotes quando crescerem.

DEBIAN — PACKAGING/DEBIAN/CONTROL (ESQUELETO)

```
Source: atena
Section: utils
Priority: optional
Maintainer: Projeto Odisseia
Build-Depends: debhelper-compat (= 13), cmake, pkg-config, libjson-c-dev, libsqlite3-dev, libcurl4-openssl-dev
Standards-Version: 4.7.0

Package: atena-core
Architecture: any
Depends: ${shlibs:Depends}, ${misc:Depends}
Description: Atena AI core for Athenas OS
```

Não congele números de Standards-Version ou dependências sem validar na distribuição alvo. O esqueleto mostra o papel dos campos; a versão final deve ser testada no Debian base do Athenas OS.

Laboratório 39 - Integrar o pacote à build do Athenas OS

OBJETIVO PRÁTICO: Fazer o live-build instalar Atena como qualquer outro pacote do sistema.

No ciclo de desenvolvimento, mantenha um repositório APT local/testing ou inclua o .deb de forma controlada na build. A ISO não deve compilar a Atena “na unha” dentro do chroot. Build do software e composição da imagem são fases diferentes.

PIPELINE

```
Atena source
  ↓ CMake/GCC/tests
Pacote .deb
  ↓ repositório testing
Athenas live-build
  ↓
ISO com Atena instalada
```

Laboratório 40 - CI e matriz de testes

OBJETIVO PRÁTICO: Fazer cada commit responder se a plataforma ainda compila, testa e empacota.

Pipeline sugerido: formatação/análise estática; GCC e Clang; unit tests; sanitizers; schema tests; pacote Debian limpo; Lintian; integração em container/VM; hashes/SBOM; assinatura apenas em job protegido. Builds de ARM64 entram depois que amd64 estiver estável.

Camada	Teste mínimo
Request/Policy	tabelas de casos e invariantes
Provider mock	streaming determinístico
llama_http	servidor de teste, timeout, JSON inválido
SQLite	migração, foreign keys, exclusão em cascata
RAG	recall de trechos e locator
Tools	schema, autorização, efeito e erro
Package	instala, inicia, remove sem resíduos críticos
ISO	boot, serviço de usuário e onboarding

Laboratório 41 - ARM64 e RISC-V sem promessas vazias

OBJETIVO PRÁTICO: Separar “compilou” de “é suportado”.

C/C++ é compilado por arquitetura. Schemas, prompts e assets podem ser Architecture: all. Cross-build e QEMU ajudam, mas uma arquitetura só vira suporte real depois de testes em placas selecionadas, benchmark, periféricos e documentação de limitações.

CRITÉRIO DE SUPORTE: Publique matriz por arquitetura e classe de hardware. “O binário foi gerado” não significa que o modelo cabe, que o áudio funciona ou que o usuário terá uma experiência aceitável.

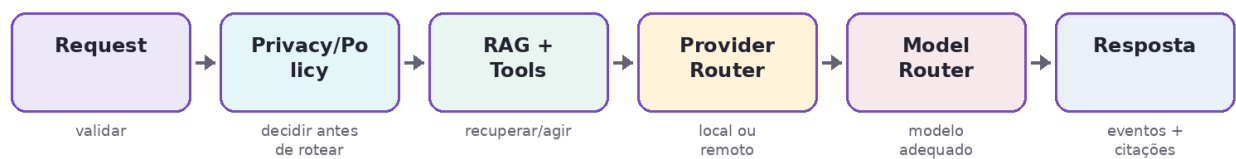
PARTE X - CAPSTONE

Atena mínima utilizável de ponta a ponta

Laboratório 42 - Montar a primeira versão utilizável

OBJETIVO PRÁTICO: Reunir os componentes mínimos em um caminho vertical completo e auditável.

Fluxo prático da Atena mínima utilizável



A versão capstone não precisa controlar qualquer aplicativo nem ter uma loja de agentes. Ela precisa demonstrar que a arquitetura não era apenas desenho: o mesmo request atravessa política, conhecimento, provider, tools e eventos sem atalhos privilegiados.

Componente	Definition of Done prática
atena-core	serviço de usuário, sem root, recebe requests por IPC
atena-cli ask	streaming e cancelamento
providers	mock + llama-server local
modos	Offline e Local Conectado no mínimo
Privacy Engine	cinco classes testadas
SQLite	conversas e preferências com migração
RAG Lite	FTS5, Markdown/TXT e locators
Citation Engine	eventos de citação por documento/trecho
tools	system.info.read + uma escrita com confirmação
Model Manager	registra, verifica e benchmarka GGUF
pacote	instala/remove e entra em ISO de teste

Laboratório 43 - Sequência recomendada de commits

OBJETIVO PRÁTICO: Transformar o livro em um histórico de engenharia fácil de revisar e reverter.

- M0.1 estrutura, CMake e status

- M0.2 request + buffer + eventos
- M0.3 provider API + mock + CLI
- M0.4 testes + sanitizers
- M0.5 policy + SQLite + schemas
- M1.1 build/health do llama-server
- M1.2 provider llama_http síncrono
- M1.3 streaming + cancelamento
- M1.4 profiler + Model Manager + benchmark
- M1.5 routers
- M2.1 FTS5 + ingestão
- M2.2 Citation Engine
- M2.3 Tool Registry + system.info.read
- M2.4 confirmação + helper privilegiado
- M3.1 serviço de usuário + IPC
- M3.2 UI cliente
- M4 pacote Debian + ISO
- M5 ARM64 / RISC-V Lab

Commits pequenos não são burocracia. Eles criam pontos de restauração. Se a implementação de streaming introduzir um leak, você consegue comparar com o commit em que o HTTP síncrono funcionava.

Laboratório 44 - Como estudar enquanto programa

OBJETIVO PRÁTICO: Usar cada erro como microaula de C, Linux ou arquitetura, sem interromper o projeto por meses.

Se aparecer...	Estude exatamente...
segmentation fault	ponteiros, lifetime, GDB e ASan
undefined reference	linker, bibliotecas e target_link_libraries
header not found	include paths, pacotes -dev e target_include_directories
JSON inválido	ownership de json-c, encoding e validação
timeout	I/O bloqueante, libcurl, cancelamento
database is locked	transações, busy timeout e concorrência SQLite
permission denied	privilégio mínimo, Polkit, paths e usuário
UI congelando	event loop, thread principal e async IPC

MÉTODO DO LIVRO: Você aprende C dentro da Atena: struct quando modelamos domínio; ponteiro quando precisamos de ownership; function pointer quando criamos Provider API; malloc quando buffers precisam crescer; processos/IPC quando o Core vira serviço. A teoria chega com endereço e propósito.

APÊNDICES

Diagnóstico, comandos rápidos, glossário e referências de atualização

Apêndice A - Diagnóstico rápido

Sintoma	Comando	Interpretação inicial
gcc não existe	which gcc; dpkg -l build-essential	toolchain não instalada
CMake não acha json-c	pkg-config --cflags --libs json-c	pacote -dev ou pkg-config
linker reclama curl	pkg-config --libs libcurl	faltou target_link_libraries
segfault	gdb --args ./build/atena-cli ...	rode e use bt
ASan acusa heap-use-after-free	build-asan + stack trace	ownership/lifetime
llama-server inacessível	ss -ltnp grep 18080	processo/porta/host
modelo OOM	free -h; dmesg tail	modelo/contexto grande demais
systemd user falha	journalctl --user -u atena-core	unit/path/permissão
SQLite lock	sqlite3 banco ".timeout 3000"	concorrência/transação

Apêndice B - Comandos de bancada

TERMINAL

```
cd ~/Projetos/atena
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure
./build/atena-cli "Olá"
```

TERMINAL

```
cmake -S . -B build-asan -DCMAKE_C_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"
cmake --build build-asan -j2
ctest --test-dir build-asan --output-on-failure
```

TERMINAL

```
git status
git diff
git add .
git commit -m "mensagem curta do marco"
git log --oneline --decorate -10
```

TERMINAL

```
systemctl --user status atena-core
journalctl --user -u atena-core -f
ss -lx | grep atena
ss -ltn | grep 18080
```

Apêndice C - Estrutura-alvo do repositório

ÁRVORE FINAL

```

atena/
├── CMakeLists.txt
├── include/atena/
│   ├── status.h      request.h      event.h
│   ├── buffer.h      provider.h     policy.h
│   ├── hardware.h    model.h        tool.h
│   └── rag.h          memory.h       ipc.h
├── core/
│   ├── request.c      policy.c        router.c
│   ├── context.c      agent.c         audit.c
│   └── ipc_server.c
├── providers/
│   ├── mock.c          llama_http.c    http_common.c
│   └── optional_cloud_adapters/
├── memory/
│   ├── store.c          migrations/
├── rag/
│   ├── chunk.c          fts.c           citations.c
├── tools/
│   ├── registry.c       system_info.c  brightness.c
├── cli/main.c
├── ui/
├── schemas/
├── voice/
├── tests/
├── docs/adr/
└── packaging/debian/
    
```

Apêndice D - Glossário de programação

Termo	Explicação prática
ABI	contrato binário entre bibliotecas/objetos; layout errado quebra plugins
API	contrato que outro módulo pode chamar
callback	função passada como dado; AtenaEventCallback é um exemplo
heap	memória de ciclo de vida explícito; malloc/free
stack	frames de função e variáveis automáticas
ownership	quem é responsável por manter e liberar um objeto
vtable	struct de ponteiros de função que simula interface substituível em C
ELF	formato comum de executável/binário em Linux
linker	combina objetos e resolve símbolos entre bibliotecas
provider	adapter entre Atena e runtime/API de modelo
runtime	programa/biblioteca que executa o modelo
GGUF	formato de arquivo de modelo usado pelo ecossistema llama.cpp
RAG	recuperar evidências antes da geração
FTS5	mecanismo de busca textual do SQLite
IPC	comunicação entre processos
Polkit	autorização de operações privilegiadas com escopo definido

Apêndice E - Regras de segurança que não podem desaparecer

- O Core roda como usuário; root só existe em helper mínimo durante ação específica.
- Nunca execute texto de shell produzido pelo modelo.
- Privacidade e autorização são aplicadas antes do roteamento de provider/modelo.
- Segredos não entram em prompt, log, screenshot, linha de comando ou repositório.
- Documentos RAG são dados não confiáveis; podem conter prompt injection.
- Tools possuem nome, schema, risco, autorização e implementação conhecida.
- Rede remota é opt-in conforme modo e classe de privacidade.
- Cancelamento, limite de bytes, timeout e número máximo de passos são parte dos contratos.
- Migrações de banco são versionadas e testadas; configuração e memória são exportáveis/limpáveis.
- “Compilou na arquitetura” não é sinônimo de “suporte publicado”.

Apêndice F - Referências e estado de atualização

Arquitetura-base: “ATENA - Engenharia de uma Inteligência Artificial Nativa para Linux”, Projeto Odisseia | Athenas OS, edição 0.1, agosto de 2026. Este livro prático reorganiza essa arquitetura em uma sequência de laboratórios de programação.

llama.cpp: comandos de build com CMake, alvo llama-server, binário build/bin/llama-server e API HTTP/OpenAI-compatible foram conferidos na documentação oficial do repositório [ggml-org/llama.cpp](https://github.com/ggml-org/llama.cpp) em agosto de 2026. Como o projeto muda rapidamente, a Atena deve fixar uma revisão testada por release e guardar a revisão no build/package.

Linux Mint: a bancada usa APT e pacotes de desenvolvimento disponíveis na base Ubuntu/Mint. Em versões futuras, confirme nomes de pacotes -dev antes de automatizar a instalação.

Debian/Athenas OS: empacotamento, systemd, D-Bus, Polkit e paths XDG devem ser validados contra a base Debian adotada no release do Athenas OS, não apenas contra a máquina Mint de desenvolvimento.

Checklist final do leitor

- Consigo explicar por que Atena não é o LLM.
- Consigo compilar C17 no Mint sem depender do botão Build do Geany.
- Tenho um repositório com CMake, testes e Git.
- AtenaRequest carrega privacidade, rede e tools além do texto.
- Provider Mock e provider llama_http implementam o mesmo contrato.
- Minha CLI troca de provider sem reescrever o Core.
- Policy Engine bloqueia cloud antes do Router.
- SQLite usa migrações e caminhos XDG.
- RAG Lite encontra trechos locais com FTS5 e devolve locators.
- Tools são funções tipadas; não existe shell genérico.
- O Core roda como usuário e ações privilegiadas são mínimas/confirmadas.
- O serviço possui IPC; CLI/UI são clientes do mesmo Core.
- O pacote .deb é a unidade de integração com o Athenas OS.
- Tenho benchmarks e uma matriz real antes de declarar suporte de hardware.

FIM DO LIVRO, INÍCIO DO PRODUTO: Se os itens acima são verdadeiros e o capstone instala e roda em uma VM limpa, você já possui a fundação de uma plataforma de IA de sistema operacional. O próximo trabalho não é “adicionar tudo”; é ampliar uma fatia por vez sem quebrar contratos, segurança e observabilidade.